

МОДУЛЬ 5 · УРОК 5.1

Помощники-функции

Научимся выносить полезный код в отдельных «помощников» и вызывать их сколько угодно раз.

C++ с нуля · 45 минут

**Один раз написал — сто раз
позвал.**

Сегодня соберём команду помощников для школьных задач.

Вспомним прошлый урок

- Цикл `for` повторяет действия нужное число раз
- В цикле удобно перебирать числа и копить сумму
- `cin` спрашивает данные, `cout` выводит ответ

Сегодня научимся **не повторять** один и тот же код, а звать готового помощника.

План урока

- 1 Зачем нужны функции
- 2 Как объявить свою функцию
- 3 Параметры — данные для функции
- 4 `return` — функция возвращает ответ
- 5 **Практика в классе** — помощники для школы
- 6 **Домашнее задание**

Зачем нужны функции?

Представь: надо поздороваться с Аней, Борей и Викой.



main.cpp

```
std::cout << "Привет, Аня!\n";  
std::cout << "Привет, Боря!\n";  
std::cout << "Привет, Вика!\n";
```

Код почти одинаковый. **Функция** — это помощник: научим его здороваться один раз и будем звать по имени.

Функция — это помощник

Без функции:

один и тот же код повторяется снова и снова

С функцией:

написал помощника **один раз** — зовёшь его по имени

Меньше кода — меньше ошибок. И программу легче читать.

Объявляем свою функцию

```
main.cpp

void pozdorovatsya() {
    std::cout << "Привет, класс!\n";
}
```

Это помощник по имени `pozdorovatsya`. Внутри `{ }` — что он делает.

- `void` — помощник **ничего не возвращает**, просто действует

Как позвать функцию



main.cpp

```
int main() {  
    pozdorovatsya();  
    pozdorovatsya();  
    return 0;  
}
```

Имя функции со скобками `()` — это вызов. Позвали дважды — поздоровались дважды.

Функцию пишем **выше** `main`, чтобы `main` уже знал про помощника.

Разбираем по строчкам

```
main.cpp

void pozdorovatsya() {           // объявили помощника
    std::cout << "Привет!\n";    // что он делает
}

int main() {                     // главная функция
    pozdorovatsya();             // зовём помощника
    return 0;
}
```

Сначала объявляем помощника, потом зовём его из `main`.

Параметры — данные для помощника

А если здороваться по имени? Передадим имя в скобках.



```
main.cpp

void privet(std::string imya) {
    std::cout << "Привет, " << imya << "!\n";
}
```

`std::string imya` в скобках — это **параметр**. При вызове кладём туда нужное имя.

Зовём с разными именами

```
main.cpp

int main() {
    privet("Аня");
    privet("Боря");
    return 0;
}
```

Выведет:

```
Привет, Аня!
Привет, Боря!
```

11

Один помощник — а здоровается с кем угодно!

`return` — ПОМОЩНИК ВОЗВРАЩАЕТ ОТВЕТ

Функция умеет не только действовать, но и считать и вернуть результат.



```
main.cpp

int ploshad(int a, int b) {
    return a * b;
}
```

Вместо `void` пишем тип ответа — здесь `int`. `return` отдаёт результат обратно.

Используем ответ функции

```
main.cpp

int main() {
    int s = ploshad(4, 3);
    std::cout << "Площадь: " << s << "\n";
    return 0;
}
```

Функция посчитала `4 * 3` и вернула 12 — мы сохранили его в `s`.

`ploshad(4, 3)` можно даже сразу вставить в `cout`.

Частые ошибки



Забыл `()`

`privet` без скобок — это не вызов. Пиши `privet();`.



Забыл `return`

Функция с типом `int` **обязана** вернуть число.



Функция ниже `main`

`main` не знает про помощника. Пиши его **выше**.

Проверь себя · что выведет программа? 🧠

```
main.cpp

int dvazhdy(int x) {
    return x + x;
}

int main() {
    std::cout << dvazhdy(5);
}
```

Подумай 10 секунд...

Выведет **10**. Функция взяла `x = 5` и вернула `5 + 5`.

Проверь себя · ещё раз 🧠

```
main.cpp

void krik(std::string s) {
    std::cout << s << "!!!\n";
}

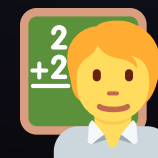
int main() {
    krik("Ура");
}
```

Что появится на экране?

Выведет **Ура!!!** — помощник добавил три восклицательных знака к слову.

ПРАКТИКА В КЛАССЕ

Собираем помощников



Пишем функции для школьных задач. Готовый код — на экране, меняем под себя.

♦ ЗАДАНИЕ

Задача 1 · Помощник-приветствие 🙌

Сделай функцию, которая здоровается по имени, и позови её трижды:

```
main.cpp

void privet(std::string imya) {
    std::cout << "Привет, " << imya << "!\n";
}
```

- В `main` позови `privet` с тремя именами одноклассников
- Запусти и проверь все три приветствия

Время: 5 минут

♦ ЗАДАНИЕ

Задача 2 · Площадь прямоугольника



Сделай функцию `ploshad`, которая считает площадь:

```
main.cpp

int ploshad(int a, int b) {
    return a * b;
}
```

- Выведи площадь доски `4 × 3` и окна `2 × 5`
- Подсказку считай: площадь = ширина × высота

Подсказка: ответ функции сразу клади в `std::cout`

✦ ЗАДАНИЕ

Задача 3 · Помощник-периметр

По образцу площади сделай функцию `perimetr(a, b)`, которая возвращает **периметр** прямоугольника.

- Формула периметра: $2 * (a + b)$
- Посчитай периметр для парты 120×60

Время: 6 минут

✦ ЗАДАНИЕ

Задача 4 · Школьный помощник

★ со звёздочкой

Сделай функцию `summa3(a, b, c)`, которая возвращает сумму **трёх** оценок ученика.

- 1 Объяви функцию с тремя параметрами
- 2 Верни их сумму через `return`
- 3 Выведи сумму оценок `5 4 5`

Кто добавит ещё функцию для среднего балла – покажем классу!

Что мы сегодня научились 🦾

- Функция — помощник: пишем код один раз, зовём много раз
- Вызов функции — это её имя со скобками `()`
- Параметры в скобках передают данные внутрь
- `return` возвращает ответ из функции
- `void` — функция действует, но ничего не возвращает

Домашнее задание

1. Реши рабочий лист 5.1 — собери свою команду помощников.
2. Загляни в шпаргалку 5.1 — как объявить и позвать функцию.

Дальше: урок 5.2 — научимся хранить целый список оценок класса.